UDB VIRTUAL
Universidad Don Bosco, El Salvador

RUTA DE APRENDIZAJE

DESARROLLO DE APLICACIONES WEB CON SOFTWARE INTERPRETADO EN EL SERVIDOR DWS901

COMPETENCIA

Desarrolla aplicaciones basadas en tecnologías emergentes: computación en la nube, Frameworks para desarrollo, adoptadas en el mercado nacional e internacional.

Realiza mantenimiento de actualizaciones de las aplicaciones web y de escritorio, según las necesidades de las instituciones.

INDICADORES DE LOGRO

- Documenta todas las etapas del desarrollo de aplicaciones que utilizan tecnología emergente: levantamiento de requerimientos, análisis, diseño, construcción, prueba e implementación de una aplicación.

- Efectúa mejoras operativas que aporten más valor a las aplicaciones web y de escritorio que utilizan tecnología emergente, teniendo en cuenta nuevos sistemas operativos o nuevos terminales que traigan consigo actualizaciones importantes.

- Crear aplicaciones basadas en tecnologías emergentes (computación en la nube, Frameworks para desarrollo) atendiendo los requerimientos de los

usuarios y los estándares relacionados con el desarrollo de software.

- Modifica las aplicaciones web y de escritorio que utilizan tecnología emergente después de entregada para corregir errores, mejorar el rendimiento y otros atributos.

FECHAS IMPORTANTES:

Mes	Actividades y fechas para entregar	Porcentaje de evaluación
Julio - Agosto	● **Unidad 1:** 20 julio - 16 agosto	
Agosto - Septiembre	● **Unidad 2**: 17 agosto - 13 septiembre ● **Proyecto fase 1**: 12 o 13 de septiembre	20%
Septiembre - Octubre	● **Unidad 3:** 14 septiembre - 11 octubre ● **Proyecto Fase 2:** 10 u 11 de octubre	25%
Octubre - Noviembre	● **Unidad 4:** 12 octubre - 08 noviembre ● **Proyecto Fase 3:** 07 u 08 de noviembre	25%
Noviembre - Diciembre	● **Unidad 5:** 09 noviembre - 05 diciembre ● **Proyecto Fase 4:** 05 diciembre	30%

Experiencia de aprendizaje: Proyecto de cátedra

Cada equipo de estudiantes asumirá el rol de una empresa desarrolladora de software encargada de analizar, diseñar, desarrollar, probar e implementar una aplicación web empresarial que responda a una problemática real o a una oportunidad de mejora identificada en una organización, emprendimiento o contexto social.

A lo largo del ciclo, el proyecto evolucionará de manera incremental mediante cuatro fases. Cada entrega representará un incremento verificable del producto y cada defensa constituirá un Sprint Review orientado a demostrar avances, justificar decisiones técnicas, recibir retroalimentación y registrar solicitudes de cambio.

El proyecto integrará progresivamente levantamiento de requerimientos, diseño de experiencia de usuario, modelado de datos, programación orientada a objetos, Laravel, Node.js, gestión de rutas y peticiones, persistencia en MySQL, autenticación, operaciones CRUD, carga de archivos, reportes, manuales, pruebas y despliegue. Al finalizar, cada equipo habrá construido un producto de software funcional, documentado y preparado para su uso en un entorno de producción.

Marco de trabajo del proyecto

Rol de los equipos	Empresa de desarrollo responsable de transformar una necesidad en una solución web empresarial funcional.
Rol del docente	Cliente, Product Owner y asesor técnico que valida el alcance, prioriza necesidades, formula observaciones y verifica la calidad.
Entregas / Defensas	Incrementos organizados en cuatro fases de proyecto (Sprints), demostrados mediante Sprint Review.
Correcciones	Solicitudes de cambio y refinamiento que deberán registrarse, priorizarse e incorporarse al incremento posterior.

Fases del proyecto

Fase	Incremento	Valor	Resultado clave
1	Descubrimiento y Arquitectura del Sistema (Sprint I)	20%	Documento Maestro del Proyecto
2	Desarrollo Back-End y Persistencia (Sprint II)	25%	Núcleo funcional y base de datos
3	Integración Full Stack y Release Candidate (Sprint III)	25%	Sistema integrado y funcional
4	Refinamiento, Despliegue y Puesta en Producción (Sprint IV)	30%	Sistema publicado y documentado

Resultados esperados

- Identificar una problemática y convertirla en requerimientos funcionales y no funcionales verificables.
- Diseñar la arquitectura, experiencia de usuario, modelo de datos y flujo de navegación de una aplicación web.
- Construir el Back-End con Laravel o Node.js, programación orientada a objetos, rutas, controladores y persistencia en MySQL.
- Integrar autenticación, operaciones CRUD, validaciones, carga de archivos, reportes y componentes de visualización.
- Aplicar pruebas, correcciones, control de versiones, documentación y mejora continua.
- Desplegar la aplicación en un servicio de hosting o nube y demostrar su funcionamiento en producción.

Modalidad

Todas las fases del proyecto serán defendidas a través de una conferencia virtual

Agrupamiento

Todas las fases del proyecto serán elaboradas y defendidas en grupos de trabajo de 5 o 6 estudiantes.

Instrumento de evaluación

Rúbrica de evaluación específica para cada fase del proyecto.

Descripción

Cada fase se detalla en los anexos de esta ruta. Los anexos establecen el objetivo, la ruta de trabajo, el producto esperado, la entrega y los criterios de evaluación.

ANEXOS

UDB VIRTUAL
Universidad Don Bosco, El Salvador

EXPERIENCIA DE APRENDIZAJE

Proyecto Fase 1 (20%)

Agrupamiento: 5 o 6 estudiantes

Fecha de entrega: 12 o 13 de septiembre

Proyecto fase 1 (20%)

Descubrimiento y Arquitectura del Sistema (Sprint I)

PONDERACIÓN	20%
MODALIDAD	Actividad grupal - demostración en documento alojado

Objetivo del Sprint

Analizar una problemática real o una oportunidad de mejora y convertirla en una propuesta formal de sistema web, documentando los requerimientos, usuarios, alcance, arquitectura, modelo de datos y experiencia de navegación que orientará todo el desarrollo posterior.

Durante esta fase, el equipo asumirá el rol de analistas funcionales, arquitectos de solución y diseñadores UX. No se implementará todavía la lógica completa del sistema; el propósito será reducir la incertidumbre y construir una base verificable para el desarrollo.

Ruta de trabajo

1. **Definición del proyecto**
    - Seleccionar una problemática real o una oportunidad de mejora que pueda resolverse mediante una aplicación web.
    - Definir nombre del sistema, contexto, justificación y propuesta de valor.
    - Identificar organización, área o usuarios beneficiados.
    - Redactar objetivo general y objetivos específicos.
    - Establecer el alcance inicial, las exclusiones y restricciones del proyecto.

2. **Levantamiento y análisis de requerimientos**
    - Identificar actores, perfiles y necesidades de los usuarios.
    - Documentar requerimientos funcionales y no funcionales.

- Redactar historias de usuario con criterios de aceptación.
- Priorizar el Product Backlog en funcionalidades indispensables, importantes y deseables.
- Definir reglas de negocio, supuestos y dependencias.

3. **Diseño funcional y experiencia de usuario**

- Definir módulos, roles, permisos y acciones principales.
- Elaborar mapa de navegación y flujos de usuario.
- Diseñar mockups o prototipo navegable de inicio de sesión, dashboard, formularios, tablas, reportes y módulos principales (herramienta para elaborar mockups).
- Aplicar principios de consistencia visual, usabilidad y retroalimentación al usuario con alertas.

4. **Arquitectura y modelado de datos**

- Proponer una arquitectura organizada por capas o patrón MVC.
- Definir entidades, atributos, relaciones y cardinalidades.
- Elaborar diagrama entidad-relación y diccionario inicial de datos.
- Definir estructura inicial del proyecto, tecnologías, dependencias y repositorio Git.

5. **Planificación del desarrollo**

- Distribuir responsabilidades dentro del equipo de trabajo.
- Identificar riesgos técnicos y medidas de mitigación.
- Preparar el Documento Maestro que servirá como fuente oficial del proyecto.

Nota

El Documento Maestro aprobado será la línea base del proyecto. Cualquier cambio posterior en requerimientos, módulos, reglas de negocio o diseño deberá documentarse y justificarse como una solicitud de cambio.

Producto esperado

- Documento Maestro en PDF alojado por un integrante al espacio definido en el aula digital, con portada e integrantes del equipo que incluya todo lo establecido en la ruta de trabajo con el enlace a la herramienta del prototipo navegable de todas las vistas de cada módulo del sistema a nivel de mockups más el enlace al repositorio Git del proyecto y el enlace de la herramienta donde estén gestionando el Product Backlog (Trello, Asana, Clickup, etc.).

Entrega (Sprint Review)

- Defensa virtual del análisis de la solución planteada, prototipo y arquitectura propuesta.
- Demostración del flujo de navegación y explicación de cómo cada módulo responde a los requerimientos.

Rúbrica de evaluación - Proyecto fase 1 (20%)

Criterios de evaluación	Inicial 0.00–3.99	Básico 4.00–5.99	Competente 6.00–7.99	Destacado 8.00–8.99	Estratégico 9.00–10.00
1. Fundamentación, alcance y propuesta de solución (15%)	No define claramente la problemática, sus causas, usuarios beneficiados ni el propósito de la solución. Los objetivos, alcance y propuesta de valor son inexistentes, contradictorios o no guardan relación con la necesidad identificada.	Identifica una problemática y propone una solución general; sin embargo, la justificación, los beneficiarios, los objetivos, el alcance o las restricciones presentan poca profundidad, ambigüedades o inconsistencias.	Define claramente la problemática, su contexto, usuarios beneficiados, objetivos y propuesta de valor. Establece un alcance inicial coherente, incluyendo las principales restricciones y exclusiones del proyecto.	Construye una fundamentación sólida que relaciona problema, contexto, necesidades, beneficiarios, objetivos, alcance y propuesta de valor, justificando de manera consistente la pertinencia de la solución planteada.	Presenta un análisis integral, crítico y verificable de la problemática, delimitando con precisión objetivos, alcance, exclusiones, restricciones y propuesta de valor. La solución evidencia viabilidad, diferenciación y una orientación clara hacia resultados medibles.
2. Especificación funcional, requerimientos y planificación del producto (25%)	La especificación funcional es incompleta, ambigua o contradictoria. No identifica claramente los módulos, funcionalidades, roles, requerimientos, reglas de negocio ni criterios de aceptación necesarios para orientar el desarrollo.	Documenta módulos y requerimientos básicos, pero existen funcionalidades omitidas, descripciones superficiales o inconsistencias entre roles, historias de usuario, reglas de negocio, criterios de aceptación y Product Backlog.	Describe los módulos y funcionalidades principales del sistema, documentando requerimientos funcionales y no funcionales, roles, historias de usuario, criterios de aceptación, reglas de negocio y un Product Backlog organizado y priorizado.	Construye una especificación funcional completa, consistente y trazable, detallando por cada módulo sus funcionalidades, actores, permisos, reglas, validaciones, dependencias, historias de usuario y criterios de aceptación. El Product Backlog está correctamente priorizado y preparado para orientar el desarrollo.	Desarrolla una especificación funcional integral, rigurosa y verificable que cubre la totalidad del sistema. Establece trazabilidad entre problema, requerimientos, módulos, funcionalidades, roles, reglas de negocio, historias de usuario, criterios de aceptación y prototipo, reduciendo significativamente la ambigüedad para las fases de construcción.
3. Prototipado integral, interfaz y experiencia de usuario (25%)	El prototipo es incompleto, confuso o no representa adecuadamente los módulos, funcionalidades, roles y flujos principales del sistema. La interfaz carece de consistencia y no permite validar la experiencia de usuario propuesta.	Presenta mockups de algunas pantallas y funcionalidades, pero existen módulos, acciones, vistas o flujos sin representar. La navegación, jerarquía visual, usabilidad o retroalimentación al usuario presentan inconsistencias.	Diseña un prototipo navegable que representa los módulos, pantallas, formularios, tablas, reportes y flujos principales del sistema. La interfaz mantiene una organización visual adecuada y permite comprender la interacción general del usuario.	Construye un prototipo integral, navegable y visualmente consistente que cubre todas las funcionalidades y módulos definidos. Representa flujos diferenciados por usuario, acciones principales, validaciones, alertas y estados de interacción, ofreciendo	Diseña una experiencia de usuario integral y profesional, completamente trazable con los requerimientos del sistema. El prototipo representa todas las vistas, módulos, funcionalidades, roles, permisos, rutas y estados relevantes, aplicando principios sólidos de UI, usabilidad, prevención

Criterios de evaluación	Inicial 0.00–3.99	Básico 4.00–5.99	Competente 6.00–7.99	Destacado 8.00–8.99	Estratégico 9.00–10.00
				una experiencia clara y coherente.	de errores y retroalimentación para orientar directamente la implementación funcional.
4. Arquitectura de solución y modelado de datos (20%)	No presenta una arquitectura técnica ni un modelo de datos funcional. Las entidades, relaciones, tecnologías o componentes son inexistentes, incoherentes o no responden a los requerimientos del proyecto.	Propone una arquitectura y un modelo de datos básicos, pero presenta inconsistencias en la separación de responsabilidades, entidades, atributos, relaciones, cardinalidades, tecnologías o estructura inicial del proyecto.	Define una arquitectura organizada por capas o patrón MVC y desarrolla un modelo entidad-relación adecuado para las principales funcionalidades. Incluye entidades, atributos, relaciones, cardinalidades, diccionario inicial de datos y tecnologías de desarrollo.	Integra una arquitectura coherente y justificable con un modelo de datos completo, normalizado y alineado con los requerimientos. Define responsabilidades por capa, estructura del proyecto, dependencias, repositorio Git y un diccionario de datos técnicamente consistente.	Diseña una base técnica integral, escalable y trazable que conecta requerimientos, arquitectura, módulos y modelo de datos. Justifica las tecnologías y dependencias seleccionadas, define con precisión componentes, responsabilidades, estructura del proyecto y repositorio, y presenta un modelo de datos robusto preparado para la implementación.
5. Documento Maestro, trazabilidad y Sprint Review (15%)	El Documento Maestro es incompleto, desorganizado o presenta contradicciones entre sus apartados. La defensa evidencia desconocimiento del proyecto y no demuestra adecuadamente el prototipo, arquitectura o decisiones propuestas.	Entrega la mayoría de los elementos solicitados, pero existen inconsistencias entre problema, requerimientos, funcionalidades, prototipo, arquitectura o planificación. La presentación y defensa son parciales o dependen de pocos integrantes.	Presenta un Documento Maestro organizado que integra los principales elementos de la fase y evidencia correspondencia básica entre análisis, requerimientos, diseño, arquitectura y planificación. Durante el Sprint Review explica adecuadamente las decisiones principales.	Entrega una documentación profesional, consistente y trazable que integra análisis, funcionalidades, prototipo, arquitectura, modelo de datos, riesgos y planificación. El equipo demuestra dominio colectivo y argumenta con claridad sus decisiones durante el Sprint Review.	Presenta una línea base integral, técnicamente sólida y altamente profesional, con trazabilidad verificable entre todos los componentes del proyecto. El Documento Maestro está preparado para orientar directamente el desarrollo, mientras que el equipo demuestra dominio integral, pensamiento crítico y capacidad para justificar, validar y defender cada decisión funcional, visual y técnica.
Logros:			**Aspectos a mejorar:**		

UDB VIRTUAL
Universidad Don Bosco, El Salvador

EXPERIENCIA DE APRENDIZAJE

Proyecto Fase 2 (25%)

Agrupamiento: 5 o 6 estudiantes

Fecha de entrega: 10 u 11 de octubre

Proyecto fase 2 (25%)

Desarrollo Back-End y Persistencia (Sprint II)

PONDERACIÓN	25%
MODALIDAD	Actividad grupal - demostración en defensa virtual

Objetivo del Sprint

Construir el núcleo funcional de la aplicación web utilizando PHP con el Framework Larvel o Node.js o programación orientada a objetos y MySQL, implementando la estructura del servidor, el manejo de peticiones, las rutas, la lógica de negocio y la persistencia de información definida en el Documento Maestro.

Durante esta fase, el equipo asumirá el rol de desarrolladores Back-End y administradores de datos. El énfasis estará en la correcta organización del código, la integridad de la información y el funcionamiento verificable de los servicios principales.

Ruta de trabajo

1. **Configuración del entorno**
    - Configurar Laravel o Node.js y el gestor de paquetes del proyecto.
    - Inicializar el proyecto y documentar dependencias.
    - Establecer una estructura organizada para rutas, controladores, modelos, servicios, vistas y archivos públicos.
    - Configurar variables de entorno y parámetros de conexión.

2. **Servidor, peticiones y ruteo**
    - Crear y ejecutar el servidor de la aplicación.
    - Implementar el manejo de peticiones y respuestas.
    - Definir rutas para los módulos priorizados.

- Aplicar middlewares básicos para procesamiento de datos, validación y manejo de errores.

3. **Programación orientada a objetos y lógica de negocio**

- Construir clases, objetos, funciones y módulos reutilizables.
- Implementar controladores o servicios para separar la lógica de negocio.
- Aplicar reglas de negocio definidas en el Sprint I.
- Gestionar eventos y operaciones asíncronas cuando corresponda.

4. **Base de datos y persistencia**

- Crear la base de datos MySQL a partir del modelo aprobado.
- Implementar tablas, migraciones, relaciones, claves primarias y foráneas.
- Configurar la conexión entre la aplicación y MySQL.
- Desarrollar operaciones CRUD para las entidades principales.
- Validar integridad, formatos y datos obligatorios.

5. **Autenticación y control de acceso**

- Implementar inicio y cierre de sesión.
- Proteger las funcionalidades de acuerdo con los roles definidos.
- Gestionar credenciales de forma segura y evitar exponer datos sensibles.
- Validar sesiones, accesos y mensajes de error.

6. **Control de versiones y pruebas iniciales**

- Registrar avances mediante commits descriptivos.
- Mantener una rama principal estable y organizar la integración del equipo.
- Realizar pruebas de conexión, rutas, consultas y operaciones CRUD.
- Documentar incidencias, limitaciones y tareas pendientes.

> **Nota**
> El incremento deberá implementar las entidades y funcionalidades prioritarias del Product Backlog. Las interfaces podrán ser básicas, pero la lógica, persistencia, autenticación y reglas de negocio deberán ser demostrables.

Producto esperado

- Aplicación ejecutable con estructura organizada.
- Base de datos MySQL funcional y script de creación o respaldo.
- Autenticación y control de acceso básico.
- Operaciones CRUD de los módulos principales.
- Repositorio Git actualizado con README de instalación y ejecución.
- Documento en PDF con portada e integrantes del equipo subido por un integrante al espacio definido en el aula digital con el enlace al repositorio Git del proyecto más el enlace de la herramienta donde estén gestionando el Product Backlog.

Entrega (Sprint Review)

- Demostración de todo lo establecido en la ruta de trabajo en defensa virtual.
- Ejecución de operaciones CRUD conectadas a la base de datos.

Rúbrica de evaluación - Proyecto fase 2 (25%)

Criterios de evaluación	Inicial 0.00–3.99	Básico 4.00–5.99	Competente 6.00–7.99	Destacado 8.00–8.99	Estratégico 9.00–10.00
1. Configuración, estructura y organización del proyecto Back-End (20%)	El proyecto no se encuentra correctamente inicializado o presenta errores significativos de configuración que impiden su ejecución. La estructura de carpetas, dependencias, variables de entorno y parámetros de conexión son inexistentes, incompletos o desorganizados.	Configura parcialmente el entorno de desarrollo y logra ejecutar el proyecto, aunque existen inconsistencias en la organización de carpetas, dependencias, variables de entorno, configuración de la conexión o documentación de instalación.	Configura correctamente Laravel o Node.js, documenta las dependencias y establece una estructura organizada para rutas, controladores, modelos, servicios, vistas y archivos públicos. El proyecto puede instalarse y ejecutarse siguiendo las indicaciones básicas del repositorio.	Implementa una estructura Back-End clara, modular y consistente, separando adecuadamente las responsabilidades de cada componente. Gestiona correctamente dependencias, variables de entorno, configuración de conexión y documentación técnica del proyecto.	Construye una base técnica profesional, mantenible y preparada para el crecimiento del sistema. La configuración, estructura modular, gestión de dependencias, variables de entorno y documentación permiten que el proyecto sea instalado, comprendido y ejecutado de manera segura y reproducible por otro equipo de desarrollo.
2. Lógica de negocio, ruteo y programación orientada a objetos (25%)	La aplicación no implementa adecuadamente el servidor, las rutas o la lógica de negocio. El código es incompleto, desorganizado o presenta errores que impiden ejecutar las funcionalidades prioritarias del sistema.	Implementa algunas rutas, controladores o funciones, pero la lógica presenta duplicación, poca separación de responsabilidades, reglas de negocio incompletas o errores en el manejo de peticiones y respuestas.	Implementa correctamente el servidor, las rutas y los controladores de los módulos principales. Aplica programación orientada a objetos, funciones reutilizables y reglas de negocio definidas durante el Sprint I, manteniendo una separación adecuada de responsabilidades.	Construye una lógica de negocio consistente y modular, integrando rutas, controladores, servicios, clases, validaciones, middlewares y manejo de errores. Las funcionalidades prioritarias responden correctamente a los requerimientos y criterios de aceptación definidos.	Desarrolla una arquitectura lógica robusta, reusable y escalable, aplicando correctamente programación orientada a objetos, separación de responsabilidades, middlewares, manejo de errores y operaciones asíncronas. La implementación mantiene trazabilidad con las reglas de negocio, historias de usuario y criterios de aceptación del Documento Maestro.
3. Persistencia, integridad y operaciones CRUD (25%)	La base de datos no se encuentra implementada correctamente o presenta errores significativos en tablas, relaciones, claves o conexión. Las operaciones CRUD son inexistentes, incompletas o no funcionan de manera verificable.	Implementa una base de datos funcional de forma parcial, aunque existen inconsistencias respecto al modelo aprobado, relaciones incompletas, validaciones insuficientes o errores en algunas operaciones CRUD.	Construye la base de datos MySQL a partir del modelo aprobado, implementando tablas, claves, relaciones y operaciones CRUD para las entidades principales. Valida datos obligatorios, formatos básicos e integridad de la información.	Implementa una capa de persistencia completa y consistente, alineada con el modelo de datos del Sprint I. Las operaciones CRUD funcionan correctamente, mantienen la integridad referencial y aplican validaciones adecuadas antes de almacenar o modificar información.	Desarrolla una solución de persistencia robusta, coherente y técnicamente justificada, con relaciones, restricciones, migraciones o scripts reproducibles, validaciones y operaciones CRUD confiables. La base de datos mantiene trazabilidad con los

Criterios de evaluación	Inicial 0.00–3.99	Básico 4.00–5.99	Competente 6.00–7.99	Destacado 8.00–8.99	Estratégico 9.00–10.00
					requerimientos, reglas de negocio y funcionalidades prioritarias del proyecto.
4. Autenticación, control de acceso y seguridad básica (15%)	No implementa autenticación funcional o permite acceder a módulos y operaciones sin los controles correspondientes. Las credenciales o datos sensibles se gestionan de forma insegura.	Implementa un inicio de sesión básico, pero existen inconsistencias en el cierre de sesión, validación de credenciales, protección de rutas, manejo de roles o mensajes de acceso denegado.	Implementa correctamente inicio y cierre de sesión, protege las funcionalidades principales y restringe los accesos según los roles definidos. Gestiona credenciales y variables sensibles sin exponerlas directamente en el código.	Configura una autenticación consistente y un control de acceso funcional, validando sesiones, roles, permisos y rutas protegidas. Los mensajes de error y acceso denegado son claros y se gestionan adecuadamente.	Implementa un esquema de autenticación y autorización técnicamente sólido, alineado con los roles y permisos del Documento Maestro. Protege rutas, sesiones, credenciales y datos sensibles mediante buenas prácticas, reduciendo accesos indebidos y riesgos básicos de seguridad.
5. Control de versiones, pruebas y Sprint Review (15%)	El repositorio se encuentra incompleto, desorganizado o no evidencia el proceso de desarrollo del equipo. No se presentan pruebas suficientes y durante la defensa existen dificultades para ejecutar o explicar las funcionalidades implementadas.	Mantiene un repositorio funcional, aunque los commits, ramas, README o evidencias de pruebas son limitados. La defensa demuestra parcialmente el funcionamiento del sistema y el dominio técnico del equipo.	Mantiene el repositorio actualizado con commits descriptivos, una estructura organizada y un README que explica la instalación y ejecución. Realiza pruebas de conexión, rutas, consultas y operaciones CRUD, documentando incidencias y tareas pendientes.	Gestiona profesionalmente el repositorio y la integración del equipo, manteniendo una rama principal estable y evidencias claras del avance. Documenta pruebas, incidencias, limitaciones y decisiones técnicas, y demuestra con claridad el incremento funcional durante el Sprint Review.	Evidencia un proceso de desarrollo colaborativo, controlado y reproducible mediante una gestión consistente de versiones, integración y documentación. Presenta pruebas verificables del servidor, rutas, base de datos, autenticación y CRUD, mientras el equipo demuestra dominio colectivo, capacidad de diagnóstico y argumentación técnica durante el Sprint Review.
Logros:			**Aspectos a mejorar:**		

UDB VIRTUAL
Universidad Don Bosco, El Salvador

EXPERIENCIA DE APRENDIZAJE

Proyecto Fase 3 (25%)

Agrupamiento: 5 o 6 estudiantes

Fecha de entrega: 07 u 08 de noviembre

Proyecto fase 3 (25%)

Integración Full Stack y Release Candidate (Sprint III)

PONDERACIÓN	25%
MODALIDAD	Actividad grupal - demostración en defensa virtual

Objetivo del Sprint

Integrar la lógica del servidor, la base de datos y la interfaz de usuario para obtener una versión completa y funcional de la aplicación web, incorporando los módulos pendientes, validaciones, gestión de archivos, reportes y elementos de visualización requeridos por el proyecto.

Durante esta fase, el equipo asumirá el rol de desarrolladores Full Stack y equipo de integración. El objetivo será convertir los componentes construidos previamente en un producto coherente, usable y preparado como Release Candidate.

Ruta de trabajo

1. **Corrección de observaciones**
    - Analizar y registrar las observaciones recibidas durante el Sprint Review anterior.
    - Corregir errores de lógica, persistencia, autenticación, estructura o experiencia de usuario.
    - Actualizar el Product Backlog y documentar los cambios realizados.

2. **Integración de la interfaz**
    - Implementar las vistas de inicio de sesión, dashboard y módulos del sistema.
    - Conectar formularios, tablas, botones y acciones con el Back-End.
    - Mantener coherencia con el prototipo aprobado.

- Construir una interfaz clara, moderna y adaptable a diferentes resoluciones.

3. Funcionalidades completas

- Completar los módulos definidos en el alcance.
- Implementar búsqueda, filtrado, edición, eliminación y visualización detallada cuando corresponda.
- Incorporar carga de imágenes o documentos, almacenando correctamente su referencia.
- Mostrar archivos cargados y permitir su consulta desde la interfaz.

4. Validaciones y manejo de errores

- Validar información en cliente y servidor.
- Aplicar expresiones regulares o reglas de formato cuando corresponda.
- Gestionar mensajes de éxito, advertencia y error.
- Restringir operaciones inválidas, registros incompletos y accesos no autorizados.

5. Reportes y visualización de datos

- Construir reportes derivados de la información almacenada.
- Incorporar totales, indicadores, balances o métricas relevantes al proyecto.
- Implementar gráficos para representar información de forma comprensible.
- Permitir la generación o exportación de reportes en PDF/Excel.

6. Pruebas de integración

- Validar los flujos completos desde la interfaz hasta la base de datos.
- Probar autenticación, roles, formularios, archivos, consultas y reportes.
- Verificar el comportamiento en diferentes navegadores y resoluciones.
- Registrar defectos, responsables, prioridad y estado de corrección.

> **Nota**
> La versión resultante será considerada Release Candidate. Deberá contener el alcance funcional acordado y servirá como base para el proceso formal de refinamiento, pruebas finales y despliegue del Sprint IV.

Producto esperado

- Sistema Full Stack completamente integrado.
- Interfaz funcional conectada con Larvavel / Node.js y MySQL.
- Autenticación, roles, módulos CRUD, archivos, validaciones y reportes implementados.
- Release Candidate disponible en el repositorio.
- Matriz de pruebas y registro de correcciones.
- Documento PDF con evidencias y enlace al repositorio.
- Documento en PDF con portada e integrantes del equipo subido por un integrante al espacio definido en el aula digital con el enlace al repositorio Git del proyecto más el enlace de la herramienta donde estén gestionando el Product Backlog.

Entrega (Sprint Review)

- Defensa virtual navegando por todos los módulos funcionales y perfiles del sistema.
- Demostración de los flujos completos, reportes, gráficos y exportaciones.
- Presentación de mejoras respecto al Sprint II y defectos pendientes para producción.

Rúbrica de evaluación - Proyecto fase 3 (25%)

Criterios de evaluación	Inicial 0.00–3.99	Básico 4.00–5.99	Competente 6.00–7.99	Destacado 8.00–8.99	Estratégico 9.00–10.00
1. Integración Full Stack y coherencia de la interfaz (20%)	La interfaz no se encuentra correctamente integrada con el Back-End o la base de datos. Existen errores significativos en formularios, navegación, consultas o acciones que impiden completar los principales flujos del sistema.	Integra parcialmente las vistas con el servidor y la base de datos; sin embargo, persisten inconsistencias en formularios, tablas, navegación, acciones o correspondencia con el prototipo aprobado.	Integra correctamente la interfaz con el Back-End y MySQL, permitiendo ejecutar los principales flujos del sistema desde las vistas. Mantiene una estructura visual coherente con el prototipo y una navegación funcional entre los módulos implementados.	Construye una solución Full Stack integrada y consistente, donde formularios, tablas, botones, acciones, autenticación y navegación se conectan correctamente con la lógica del servidor y la persistencia. La interfaz es clara, moderna y adaptable a diferentes resoluciones.	Implementa una integración Full Stack profesional, estable y completamente trazable con el prototipo y los requerimientos. La interfaz, lógica de negocio y base de datos operan de forma coordinada, ofreciendo una experiencia fluida, consistente y preparada para su validación como Release Candidate.
2. Cobertura funcional de módulos y flujos del sistema (25%)	El sistema presenta módulos o funcionalidades esenciales incompletas, inexistentes o con errores que impiden cumplir el alcance acordado. Los flujos definidos para los distintos perfiles no pueden ejecutarse de manera integral. .	Implementa parcialmente los módulos y funcionalidades del proyecto, aunque existen omisiones, acciones incompletas o inconsistencias entre el Product Backlog, los requerimientos y el comportamiento real del sistema.	Completa los módulos prioritarios y permite ejecutar sus flujos principales, incluyendo operaciones CRUD, búsqueda, filtrado, visualización y gestión de archivos cuando corresponda. La implementación responde adecuadamente al alcance definido.	Implementa de manera completa y consistente los módulos, funcionalidades y flujos comprometidos. Cada perfil puede ejecutar las acciones correspondientes, y existe trazabilidad clara entre requerimientos, Product Backlog, criterios de aceptación e incremento funcional.	Entrega una cobertura funcional integral y verificable del alcance acordado. Todos los módulos, roles, acciones, excepciones y flujos principales se encuentran implementados, integrados y alineados con el Documento Maestro, demostrando que el sistema está preparado para su liberación como Release Candidate.
3. Validaciones, manejo de errores y control de operaciones (20%)	El sistema permite registrar o procesar información inválida, incompleta o inconsistente. Los errores no se controlan adecuadamente y los mensajes al usuario son inexistentes, confusos o insuficientes.	Implementa validaciones básicas en algunos formularios o procesos, pero existen omisiones entre cliente y servidor, mensajes poco claros o reglas de negocio que pueden ser ignoradas.	Aplica validaciones en cliente y servidor para los principales formularios y operaciones. Controla formatos, campos obligatorios, accesos y reglas básicas de negocio, mostrando mensajes adecuados de éxito, advertencia y error.	Integra un esquema consistente de validación y manejo de errores, impidiendo operaciones inválidas, accesos no autorizados y registros incompletos. Los mensajes proporcionan retroalimentación clara y orientan correctamente al usuario.	Implementa un control integral y robusto de datos, reglas de negocio, accesos y excepciones. Las validaciones funcionan de forma coordinada entre cliente y servidor, previenen inconsistencias y ofrecen una experiencia segura, predecible y

Criterios de evaluación	Inicial 0.00–3.99	Básico 4.00–5.99	Competente 6.00–7.99	Destacado 8.00–8.99	Estratégico 9.00–10.00
					preparada para producción.
4. Reportes, visualización y exportación de información (20%)	No implementa los reportes requeridos o presenta información incorrecta, incompleta o desconectada de los datos almacenados. Los gráficos o exportaciones no funcionan de forma verificable.	Implementa reportes o visualizaciones básicas, aunque presentan limitaciones en cálculos, filtros, diseño, interpretación o generación de archivos PDF o Excel.	Construye reportes funcionales a partir de la información almacenada, incorporando totales, indicadores, balances o métricas pertinentes. Implementa gráficos comprensibles y al menos un mecanismo de exportación funcional.	Integra reportes completos, claros y correctamente calculados, acompañados de gráficos pertinentes y opciones de exportación a PDF o Excel. La información responde a las necesidades del proyecto y facilita su análisis.	Desarrolla un sistema de reportes integral, confiable y orientado a la toma de decisiones. Las consultas, métricas, totales, visualizaciones, filtros y exportaciones se encuentran completamente integrados, presentan información precisa y aportan valor real al usuario.
5. Pruebas de integración, gestión del Release Candidate y Sprint Review (15%)	No evidencia un proceso formal de pruebas ni una gestión clara de las observaciones y defectos. El Release Candidate presenta fallos críticos y el equipo demuestra dificultades para ejecutar o explicar el sistema durante la defensa.	Realiza pruebas parciales y documenta algunas correcciones, pero la matriz de pruebas, el registro de defectos, la actualización del Product Backlog o la identificación de la versión son insuficientes o inconsistentes.	Ejecuta pruebas de integración sobre los principales flujos del sistema, documentando casos, resultados, defectos, responsables y estado de corrección. Registra las mejoras realizadas y mantiene una versión identificable como Release Candidate en el repositorio.	Gestiona de manera organizada el proceso de integración y estabilización, manteniendo una matriz de pruebas completa, un registro actualizado de defectos y trazabilidad de las correcciones. Durante el Sprint Review demuestra el funcionamiento integral del sistema y explica con claridad los pendientes para producción.	Presenta un Release Candidate estable, verificable y técnicamente controlado. El equipo demuestra un proceso profesional de pruebas, gestión de defectos, control de cambios y versionamiento, evidenciando trazabilidad entre observaciones, Product Backlog, correcciones y resultados. Durante la defensa demuestra dominio colectivo y capacidad para justificar la preparación del sistema para el Sprint IV.
Logros:			**Aspectos a mejorar:**		

UNIVERSIDAD DON BOSCO

UDB VIRTUAL

Universidad Don Bosco, El Salvador

EXPERIENCIA DE APRENDIZAJE

Proyecto Fase 4 (30%)

Agrupamiento: 5 o 6 estudiantes

Fecha de entrega: 05 de diciembre

Proyecto fase 4 (30%)

Refinamiento, Despliegue y Puesta en Producción (Sprint IV)

PONDERACIÓN	30%
MODALIDAD	Actividad grupal - demostración en defensa virtual

Objetivo del Sprint

Refinar la aplicación web mediante la corrección de observaciones, la ejecución de pruebas finales, la optimización técnica y el despliegue en un servicio de hosting o nube, entregando una versión estable, documentada y preparada para su utilización en un entorno de producción.

Durante esta fase, el equipo asumirá los roles de aseguramiento de calidad, DevOps y gestión de liberaciones. El énfasis estará en garantizar que el sistema publicado sea funcional, estable, mantenible y comprensible para usuarios y administradores.

Ruta de trabajo

1. **Refinamiento y control de cambios**
    - Consolidar observaciones de todos los Sprint Review.
    - Clasificar correcciones funcionales, técnicas, visuales y documentales.
    - Implementar los cambios priorizados y mantener trazabilidad con los requerimientos.
    - Preparar un registro de cambios o Release Notes.

2. **Aseguramiento de calidad**
    - Ejecutar pruebas funcionales, de integración, navegación y compatibilidad.
    - Verificar autenticación, permisos, operaciones CRUD, carga de archivos y reportes.
    - Validar reglas de negocio, mensajes y manejo de errores.

- Corregir defectos críticos y documentar limitaciones conocidas.

3. **Optimización y seguridad básica**

    - Eliminar archivos, dependencias y código innecesario.
    - Optimizar consultas, recursos y tiempos básicos de respuesta.
    - Revisar validación de entradas, rutas protegidas y exposición de información sensible.

4. **Despliegue en producción**

    - Seleccionar un servicio de hosting o plataforma en la nube compatible.
    - Publicar la aplicación y configurar la base de datos de producción.
    - Configurar variables de entorno, rutas, dependencias y almacenamiento.
    - Verificar el acceso mediante una URL pública y el funcionamiento de todos los módulos.

5. **Documentación del producto**

    - Elaborar manual de usuario con instrucciones de acceso y uso de las funciones principales.
    - Elaborar manual técnico con arquitectura, requisitos, instalación, base de datos, configuración y mantenimiento.
    - Actualizar README, scripts, respaldo de base de datos y repositorio final.

6. **Cierre y presentación del proyecto**

    - Preparar la demostración final del producto publicado.
    - Comparar los resultados con los objetivos y criterios de aceptación iniciales.

> **Nota**
> La URL publicada constituirá la versión oficial del proyecto. El equipo deberá verificarla antes de la defensa y mantener disponibles los accesos, datos de prueba y recursos necesarios para la evaluación.

Producto esperado

- Documento PDF final del proyecto con portada e integrantes del equipo alojado por un integrante al espacio definido en el aula digital con el URL público funcional del sistema web, matriz de credenciales por roles y permisos, repositorio Git del proyecto, enlace de la herramienta donde se gestionó el Product Backlog, matrices de prueba, manual de usuario y manual técnico.

Entrega (Sprint Review)

- Demostración virtual de la aplicación publicada desde la URL de producción.
- Navegación por todos los módulos, perfiles, reportes y funciones principales.
- Presentación del proceso de despliegue, pruebas, optimizaciones y correcciones.

Rúbrica de evaluación - Proyecto fase 4 (30%)

Criterios de evaluación	Inicial 0.00–3.99	Básico 4.00–5.99	Competente 6.00–7.99	Destacado 8.00–8.99	Estratégico 9.00–10.00
1. Refinamiento, control de cambios y cumplimiento del alcance (20%)	No evidencia un proceso organizado de refinamiento. Las observaciones de los Sprint anteriores permanecen sin atender o se implementan cambios sin registro, prioridad ni relación con los requerimientos y criterios de aceptación del proyecto.	Corrige parcialmente algunas observaciones, aunque persisten defectos, cambios sin documentar o inconsistencias entre el alcance acordado, el Product Backlog y la versión final del sistema.	Consolida y clasifica las observaciones recibidas, implementa las correcciones prioritarias y mantiene una relación básica entre cambios, requerimientos y funcionalidades. Presenta un registro de cambios o Release Notes de la versión final.	Gestiona de forma consistente el refinamiento del sistema, priorizando y documentando correcciones funcionales, técnicas, visuales y documentales. Mantiene trazabilidad entre observaciones, Product Backlog, requerimientos, cambios implementados y resultados obtenidos.	Ejecuta un proceso integral y controlado de refinamiento y gestión de cambios, evidenciando trazabilidad completa desde las observaciones hasta la liberación final. La solución cumple el alcance y los criterios de aceptación acordados, y las Release Notes documentan profesionalmente las mejoras, correcciones y limitaciones conocidas.
2. Aseguramiento de calidad y estabilidad del sistema (25%)	No ejecuta pruebas suficientes o el sistema presenta defectos críticos que impiden completar los principales flujos. Las matrices de prueba son inexistentes, incompletas o no reflejan el funcionamiento real de la solución.	Realiza pruebas parciales sobre algunas funcionalidades, pero persisten fallos relevantes en autenticación, permisos, operaciones CRUD, archivos, reportes, navegación o reglas de negocio. La documentación de defectos y resultados es limitada.	Ejecuta pruebas funcionales y de integración sobre los principales módulos, perfiles y flujos del sistema. Documenta resultados, defectos, responsables y estados, corrigiendo los problemas críticos antes de la liberación.	Implementa un proceso de aseguramiento de calidad organizado y verificable, cubriendo autenticación, roles, operaciones, archivos, reportes, compatibilidad, navegación y manejo de errores. Las matrices de prueba evidencian una versión estable y con defectos controlados.	Presenta un proceso profesional de QA, con cobertura integral de los flujos críticos y trazabilidad entre casos de prueba, requerimientos, resultados y defectos. El sistema demuestra estabilidad, consistencia y confiabilidad, mientras las limitaciones conocidas se encuentran claramente documentadas y justificadas.
3. Optimización técnica y seguridad básica (15%)	El sistema conserva código, archivos o dependencias innecesarias y presenta deficiencias importantes en validación, protección de rutas o manejo de información sensible. La solución evidencia poca	Aplica algunas mejoras de optimización y seguridad, aunque persisten consultas ineficientes, recursos innecesarios, rutas insuficientemente protegidas o configuraciones sensibles expuestas.	Elimina elementos innecesarios, optimiza consultas y recursos principales, protege rutas y valida entradas. Gestiona adecuadamente variables de entorno y evita exponer información sensible en el código o repositorio.	Refina técnicamente la solución mediante una organización eficiente del código, dependencias, consultas y recursos. Aplica controles consistentes sobre validaciones, accesos, configuración y exposición de datos sensibles.	Entrega una solución optimizada, mantenible y preparada para producción, aplicando buenas prácticas de rendimiento y seguridad básica de forma integral. Las decisiones técnicas reducen riesgos, mejoran la eficiencia y fortalecen la

Criterios de evaluación	Inicial 0.00–3.99	Básico 4.00–5.99	Competente 6.00–7.99	Destacado 8.00–8.99	Estratégico 9.00–10.00
	preparación técnica para producción.				estabilidad operativa del sistema.
4. Despliegue y funcionamiento en producción (25%)	La aplicación no se encuentra publicada o la URL presenta errores que impiden su acceso o funcionamiento. La base de datos, rutas, dependencias, almacenamiento o variables de entorno no están configurados correctamente.	Publica la aplicación, pero existen fallos de configuración, accesibilidad, persistencia, archivos, rutas o credenciales que afectan parcialmente el funcionamiento del sistema en producción.	Despliega correctamente la aplicación y la base de datos en un entorno de hosting o nube. La URL pública permite acceder a los principales módulos, perfiles y funcionalidades del sistema.	Gestiona un despliegue estable y correctamente configurado, verificando variables de entorno, dependencias, rutas, almacenamiento, base de datos y accesos por rol. Todos los módulos principales funcionan de manera consistente desde la URL pública.	Ejecuta una puesta en producción profesional, estable y reproducible. La aplicación, base de datos, archivos, perfiles, módulos, reportes y configuraciones operan correctamente en el entorno publicado, evidenciando dominio del proceso de liberación y capacidad para mantener la solución disponible para su uso real.
5. Documentación final, cierre del proyecto y Sprint Review (15%)	La documentación final es incompleta, desorganizada o no permite comprender, instalar, utilizar o mantener el sistema. Durante la defensa, el equipo presenta dificultades para demostrar el producto publicado o explicar las decisiones tomadas.	Entrega la mayor parte de los documentos solicitados, pero existen omisiones o inconsistencias en los manuales, credenciales, pruebas, repositorio, respaldo o explicación técnica del proyecto.	Presenta una documentación organizada que incluye manual de usuario, manual técnico, README, enlaces, credenciales, pruebas y respaldo de la base de datos. Durante el Sprint Review demuestra correctamente la versión publicada y explica el proceso de cierre.	Entrega una documentación completa, clara y profesional que facilita el uso, administración, instalación y mantenimiento del sistema. El equipo demuestra dominio colectivo, compara los resultados con los objetivos iniciales y justifica las decisiones de despliegue, pruebas y optimización.	Presenta un cierre integral y altamente profesional del proyecto, con documentación técnica y funcional completa, actualizada y coherente con la versión publicada. Los manuales, matrices, repositorio, credenciales, respaldos y evidencias permiten operar y mantener la solución, mientras el equipo demuestra dominio total del ciclo de vida, pensamiento crítico y capacidad para defender la preparación del sistema para producción.
Logros:			**Aspectos a mejorar:**		